



On The Quantification of e-Business Capacity

Moises Goldszmidt
moises@peakstone.com

Derek Palma
dpalma@peakstone.com

Bikash Sabata
sabata@peakstone.com

Peakstone Corporation
155A Moffet Park Drive
Sunnyvale, CA 94089

ABSTRACT

In order for current e-Businesses to mature from hastily assembled systems and applications, formal processes must be put in place for planning and budgeting, pricing and costing, and for establishing quality of service and service-level assurances. There are many challenges that e-Businesses face in formalizing these processes. The most important problem is to bridge the semantic disconnect between business objectives and the information system performance objectives. Next, the characterization of the e-Business infrastructure is extremely complex, given the variety of applications and system configurations at a web site and the traffic it receives. Finally, e-Businesses need to associate and apply traditional economic factors, such as depreciation and usage to applications, operating systems, and databases. In this paper, we propose an approach for defining and quantifying **effective e-Business capacity** that allows us to translate quality of service objectives into the number of users that a web site can support. This approach is based on inducing online models using machine learning and statistical pattern recognition techniques. As a consequence, the approach is flexible: it adapts to any site configuration and environment. The concept of e-Business capacity allows us to naturally answer planning and operational questions about the information system infrastructure needed to support the e-Business. The questions range from indicating which performance measures in the system are “important” to simulating “if-then” scenarios.

1. INTRODUCTION

Current e-Businesses seem to struggle with the ability to quantify, control, and plan the utilization of the available capacity in their e-Business infrastructures. Few, if any, can provide a definite answer to the following question: “given that you would like to service each URL request with a response time of one second, 90% of the time, how many customers can your web site support?” This situation would be cause for great alarm in any traditional brick and mortar

business. Management techniques, models, and methodologies have long existed and are quite mature for every segment of the manufacturing and service industries. In such companies, executives make business decisions based on a precise understanding of available capacity, the capacity required to maximize different business opportunities, and the cost of unused capacity (waste). In a growing competitive marketplace, e-Businesses are under constant pressure to optimize their utilization of resources. The survival of e-Businesses parallels the same challenges faced by brick and mortar businesses. Unfortunately, in spite of these factors, the production plant of an e-Business – call it an **e-Factory** – has not, up to this date, been managed in a manner that yields the same capabilities to quantify, control, and plan capacity utilization and expansion.

To solve this problem, Information Technology (IT) system administrators and planners have tried to use methodologies derived from the performance measurement and modeling of the distributed systems. Traditionally, the techniques are used effectively to characterize the “health”¹ of the system. However, applying such techniques to address the business objectives of the infrastructure have proven insufficiencies. Consider a simple example given by the graphs in Figure 1. The x-axis represents the number of user sessions in a site. For the CPU utilization curve, the y-axis represents the utilization (as provided by the operating system metrics at the web server). Note that after 150 sessions, the CPU’s utilization is 100%. For the *response* curve, the y-axis represents the percentage of requests that take more than 1 second to be served. Note that by 250 sessions, the system is still able to provide a quality of service of 1 second per request, even though the CPU utilization is constant at 100%.

Performance modeling efforts based on queuing networks such as the one described in [10] begin to solve some of these problems in our view, fall short of providing the full set of solutions that are needed. This approach is based on building analytical models of the web site– process which requires expertise. It also requires the collection of data and the computation of input parameters for the models. This involves figuring out the service demands of each type of request on each device; a very costly enterprise. Finally, although this approach can answer questions regarding the average throughput of the system, it cannot answer the question posed above, where percentages are involved. Yet, there are undeniable advantages of having analytical models (see

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. To copy otherwise, to republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee.

EC’01, October 14-17, 2001, Tampa, Florida, USA.

Copyright 2001 ACM 1-58113-387-1/01/0010 ...\$5.00.

¹Health of the system is related to the utilization of the different resources such as CPU, memory, disk, etc.

Section 5). Simulation models may be able to deal with percentages, and a combination may be able to deal with the question above. Yet, the building and validation of simulation models is by no means a trivial exercise.

We propose an alternative, unique, and innovative approach based on machine learning and statistical pattern recognition [1, 3, 6], to compute the number of user sessions that a web site can support at a given quality of service. We name this characteristic of a web site as **Effective e-Business Capacity (EEC)**. As we will see, the quantification of EEC will enable the computation of other important business-related properties of a web site, such as overall utilization by service, trade-offs between the different components of the quality of service, trade-offs between the different services. These factors will enable appropriate planning and the appropriate understanding of the IT infrastructure in terms of business objectives. By relying on an approach based on inducing statistical models from observed workload and performance data, we sidestep the problems of model building and necessary expertise. The approach adapts to the myriad of infrastructure configurations, applications, etc., and automatically builds models that directly associate the state of the system being observed with indications of whether the quality of service is being met.

On the other hand, the approach presents new problems related to the methodology. For example, the need to consider biases introduced when selecting algorithms for induction and parameterizations for the models, accuracy measurements, problems of dimensionality, and over-fitting versus generalizing. However, there is a great deal of current research on machine learning and pattern recognition motivated by the promise of knowledge discovery and data-mining and producing computationally oriented algorithms capable of facing ever more demanding applications and domains. We include, for example, research in text classification and collaborative filtering (which require the handling of large amounts of data in high dimensional spaces), research in biological applications, robot vision and control, and many other areas.

In the next section, we introduce the economic considerations to manage the e-Business infrastructure. This provides the motivation for the definition of the Effective e-Business Capacity. In Section 3, we present the specifics of the approach including experiments illustrating the quality of the inferences about EEC. Section 4 describes the use of the EEC for capacity planning and provisioning. Finally, Section 5 concludes the paper with a short discussion.

2. BUSINESS PRACTICES TO EFFECTIVE E-BUSINESS CAPACITY

Although businesses cannot function without the ability to quantitatively manage their production facilities, e-Businesses are just beginning to realize that the same kind of quantification of e-Factory capacity is absolutely required. For effective planning and management of the infrastructure e-Businesses need to understand the:

- **Utilization of capacity.** What percentage of capacity is used by service X and/or customer Y? How much capacity is unused? When is there a shortage of capacity?
- **Cost of capacity.** What is the cost of the capacity consumed for each service and/or customer? What is the cost of unused capacity?

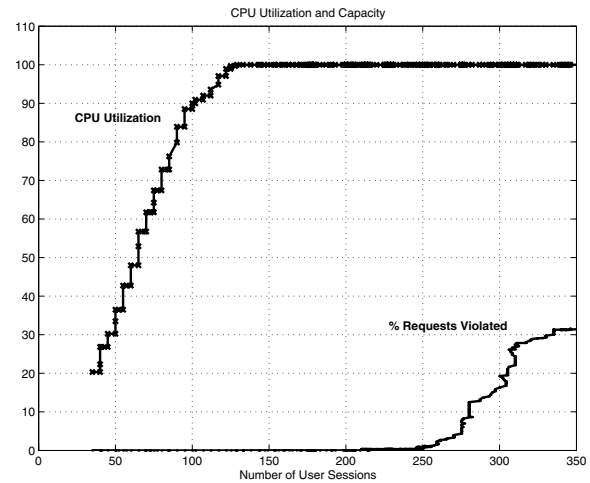


Figure 1: Failure of CPU utilization to quantify the EEC of a web site.

- **Revenue benefits.** What is the revenue benefit of applying available capacity to service X versus service Y? How can business opportunities be optimized with available capacity? What capacity allocation policies will maximize revenue and other business benefits?
- **Service quality.** What level of service quality is being delivered for each service? What are the trade-offs between service quality, cost, and volume? What capacity allocation policies will yield the desired trade-off?
- **Operations risk.** What are the current operations risk levels? Is there enough capacity available to support the expected business opportunities? What capacity allocation policies are required to achieve the desired risk levels? Can the infrastructure support the roll out of a new application?

Note that we use the aggregate notion of the *capacity of the system* and not the resource utilization measures.

When production orders arrive at brick and mortar companies, much care is given to ensuring the capacity of the available production facilities is optimally utilized for the product demand, based on business policies. Such business policies ensure that each business opportunity is maximized. Unfortunately, e-Businesses do not have this capability. When customers arrive and place demand on the e-Business infrastructure, e-Businesses are not able to effectively define implement, or enforce allocation policies (due to their inability to quantify e-Business capacity). Currently, all allocation policies are manually implemented and dependent on the physical speed and intuitive judgment of human operators. Automated techniques similar to those available in other mature industries are required. As a first step, we define a framework for quantifying the cost and benefits of the e-Factory to an e-Business.

2.1 e-Factory Cost Model

We define a *e-Factory cost model* by translating traditional cost accounting practices [11] to the e-Business. The focus of the e-Factory cost model is to quantify the cost of each type of e-Business transaction. The cost model requires the following inputs:

- **An e-Business job.** In our context, this will be a unit of work which has tangible business value. For example, the

the *user session*, consisting of a sequence of URL queries to the system to use a service, has tangible business value and therefore can be considered as an e-Business job.

- **An e-Business transaction.** The e-Business job consists of a sequence of e-Business transactions. The successful completion of the transactions results in the successful completion of the job. In our example, the URL queries are the e-Business transactions.

- **A service quality metric.** The business objectives are translated to the quality metric for the application. The quality metric must be objectively quantifiable and measurable and is most relevant to the business objectives. For example, one such metric could be the response time for each transaction in an application. The cost of delivering the same transaction at different quality levels may vary greatly and this variability impacts the *business value of the transaction (BVT)*. As a result, there exists a bound (usually a function of time) on the quality after which the transaction has diminishing value to the business.

- **A consistency quantification.** This represents the e-Business's quality tolerance. Specifically, consistency represents the percentage of transactions which meet the target quality some percentage of the time. For example, a 98% consistency would imply that, on the average, for every 100 transactions, 98 will meet their target quality for response time and 2 may not. This is similar to the notion of production line yield, where some number of defects are expected since it may not make economic sense to achieve 100% yield.

- **A quality objective.** Execution of transactions must be related to a quality objective to quantify their business utility. The quality objective specifies the target quality in terms of a metric and a target consistency as a percentage of time or volume. E.g., the business specifies the service quality of response time of the transactions is 1 second and the desired consistency is 95%. Again, considering the production line analogy, this ensures that the business value of the output is quantified.

- **A capacity utilization quantification.** In general, *capacity* is the amount of output that can be obtained from a process. In the context of an e-Business, we must focus on effective e-Business capacity (EEC), which quantifies its ability to deliver a specific volume of transactions meeting a specific quality objective.

The e-Factory cost model, using the above inputs is then able to provide detailed cost analysis of the e-Business. From the total cost of providing the e-Business infrastructure and the *capacity utilization*, we can compute the cost per transaction, cost of each service, and the cost to support each customer. We can also compute the cost of the unused capacity in the infrastructure or the value of each additional unit of capacity. Note that without the abstraction of the e-Business capacity, we will not be justified in doing the same with the resource utilization metrics. For example, consider the configuration in table 1 where the web infrastructure supports two services, *A* and *B*. The cost to the business per month is \$100,000.00 and there are 10,000 users of service *A* and 500,000 users of service *B*. Using the techniques discussed later in this paper, we determine that service *A* uses 30% of the e-Business capacity and service *B* uses 50% of the e-Business capacity. By simple computations, we can see that the cost per user for service *A* is \$3.00 and service *B* is \$0.10. The cost of the *unused capacity* or the *headroom*, can also be calculated, and prudent use of headroom is fre-

quently required to account for peak demand or a surge in demand. In this example, the cost of the unused capacity is \$20,000.00 per month.

	# Users	Capacity Utilization	Cost	Cost/ User
<i>A</i>	10,000	30%	\$30,000.00	\$3.00
<i>B</i>	500,000	50%	\$50,000.00	\$ 0.10

Table 1: The infrastructure supports two services, *A* and *B*. The cost of infrastructure per month is \$100,000.00. The simple cost analysis shows the cost per user of each service. The cost of the unused capacity is \$20,000.00 per month.

The focus of the e-Business is to minimize the cost of each transaction, maximize the potential business opportunity, and manage risk. All of these goals depend on an accurate quantification of the e-Factory's ability to support specific transaction types with specific service quality levels. **Effective e-Business Capacity (EEC)** represents this quantification.

2.2 Effective e-Business Capacity

The e-Factory cost model demands that e-Businesses define two fundamental concepts as they relate to their business: (a) the specification of the desired quality objective and (b) the quantification of the effective e-Business capacity. The quality objective requires the identification of the appropriate quality metric and the specification of its target value. The required target consistency must also be specified. The quality objective is typically a function of strategic objectives which address customer experience or competitiveness in the respective market place.

Current approaches based on resource utilization metrics for e-Businesses capacity quantification have fallen short from a business utility point of view since service quality is rarely taken into account in the models. The concept of effective e-Business capacity tries to capture the notion of the business value of the work done by the infrastructure. The quantification of EEC consists of necessary measurement analysis to determine the effective quantity of capacity available as well as determine the proportion of this capacity consumed by each transaction type. In section 3, we describe the procedure to compute this. Using the EEC of the web infrastructure, we can determine the effective costs of all transactions as well as the cost of unused capacity.

The quality objective and the EEC are intimately related. Consider the the graph in figure 7. The plot shows the trade-offs between response time, consistency, and transaction volume for a typical web site. The most important fact to observe is that the site's ability to deliver a service volume is a function of the service quality objectives (response time and consistency). The service quality objectives define the feasible operating range of the site where business value is attainable. As a site operates outside of this range due to overloading, it delivers diminishing business value. The primary operations objective for the site is to optimally utilize the available EEC.

The most important conclusion is that any quantification of EEC must consider the quality objective of the business. Failure to do this voids any practical interpretation of EEC. Unfortunately, many e-Businesses quantify their capacity without considering the service quality objectives. Due to

the resulting error, they are unable to realistically deliver on the business objectives from the site. Therefore, a fundamental requirement for the quantification of the EEC is that for a given workload, it should satisfy the following:

1. If all service objectives were met, then the computed EEC utilization of the system should be less than or equal to 100%.
2. If service objectives are not met, then the computed EEC utilization of the system should be 100%.

The traditional capacity planning approaches [10] are not easily applicable to the e-Factory cost model due to the difference of focus. In the traditional approaches, measures are typically developed from a bottom-up perspective and do not necessarily measure the core processes and critical success factors as defined in the business strategy. They are measuring the application and resource performance from the system administrator point of view who is interested in the “health” of the system. In cases where application usage and customer behavior is recorded, the focus is on the historical performance of internal operations. These methods generally report what happened last period within the organization, but provide no information on how managers can improve performance in the following period. On the financial reporting side, there is no attempt to incorporate key performance indicators as an integral part of the management reporting system. There is no link between the business objectives and the system performance in all of these planning methodologies.

3. COMPUTING EEC

The effective e-Business capacity is a characterization of an e-Business web system in terms of the number of *e-Business jobs* that can be serviced under some predefined service quality and consistency constraints. Our approach is based on inducing a set of models from data capable of characterizing the behavior of the web system in the terms expressed in the previous section. We essentially “mine” event logs collected from the different components of the web infrastructure, treating it as a black box. This section will describe in detail how these computations are performed, and will also provide experimental results pertaining to the quality of these inferences. We start with preliminary definitions directed at establishing a common language. We then describe the input data and the set of models that are induced. We follow with an assessment of the quality of these models and introduce the final algorithm.

The *basic operation* we will be concerned with is the e-Business transaction which, for the rest of the paper, is defined as the satisfaction of a URL request by the web infrastructure. This operation entails all the computations from the moment that the request arrives at the web site to the moment it departs from the web site. As a measure of quality of this service, we will use, for the purposes of this paper, the response time of this operation which will be denoted as t_r . Note that this time is measured from the moment that the request enters the web site, until it departs the site. Give a specific response time objective t_{ro} , a request will be said to be *violated* iff the time taken to satisfy this request t_r , namely its response time, is bigger than t_{ro} , i.e., iff $t_r > t_{ro}$. The complete specification of the the service quality requires the specification of the *consistency*. Formally, given a set of

requests R , the consistency of the service for R will be given by the percentage of requests in R that are not violated. Namely, by the percentage of requests whose response time is less than or equal to the requirement t_{ro} . We are now ready to introduce the definition of e-Business capacity.

DEFINITION 1. *Given a response time objective t_{ro} , a consistency requirement c , and a set of requests R . We say that the system is in capacity for R if the consistency requirement is satisfied. Otherwise, we say that the system is out of capacity.*

As an example, let R be defined as the set of requests that are active during any minute interval. Let t_{ro} be one second and let $c = 90\%$. In any given minute interval, the system will be in capacity if 90% of the active requests are serviced in less than one second. Note that this definition of capacity comprises a business view directed at establishing quality of service, rather than representing component resource utilization metrics. This later view is what is normally used in information technology-oriented performance evaluation of components. Our proposal imposes a business-oriented view. Our objective is to establish a quantitative measure of system capacity that can be used to determine how many e-Business jobs the system can support given the quality of service requirements. This is what we call effective e-Business capacity (EEC). The workload on the system depends on the e-Business jobs (which are the sessions in our case) that are active in the system. The workload generated by the sessions is the constituent e-Business transactions (which are the URL requests in our case). Defining the session is another decision the business has to make. One way to define is by associating all the requests for an application/service issued by an uniquely identified user. We are now ready for the definition of effective e-Business capacity.

DEFINITION 2. *Given a response time objective t_{ro} , a consistency requirement c , and a specification of the set of requests R , The Effective e-Business Capacity (EEC) of a web site is the number of e-Business jobs that a system can service while staying in capacity.*

Following the specifications of the example above, the EEC will be the number of sessions such that, for any minute, 90% of the requests generated by these sessions are serviced in less than one second. Note that it is straightforward to generalize this definition to the case where a set of different service quality guarantees are specified for different groups of customers (or URLs). For clarity of exposition we will maintain a unique specification in this paper.

Now we turn our attention to the computation of EEC. Our proposal rests on inducing a set of functions directly from the measurements from the various resources in a given web site. The composition of these functions will enable a user to estimate the EEC given the specification of the quality of service desired. We will, in fact, data mine the measurement logs in order to provide the necessary information for business planning and decision making.

We will use two algorithms, which in turn will require a different set of functions. As these functions incur errors introduced by the measurements, the inherent noise in the data, and the necessary abstractions and summarizations, we minimize the effect of these errors by fusing the results of these algorithms (Section 3.3).

The first algorithm, which we call *consistency regression* or **CR**, relies on inducing a “family” of *sessions v/s consistency* or **SvC** functions that map, for a given response time objective, the percentage of requests being serviced in a time less than or equal to the response time, to the number of customers in the web site.

The second algorithm, which we call *capacity boundary search* or **CBS** for short, relies on an implementation of the definition of capacity introduced above. Using other functions that map the number of sessions to the workload they generate, the algorithm simply increases the number of sessions until the capacity function establishes that the system is out of capacity.

As is evident, the approach rests on having a set of functions describing the behavior of the web site. In our proposal these functions are induced directly from measurements at different resources in the web site. These measurements collect performance data from the operating system (and database), and workload information from the entry point in the site. There are essentially three parts to the whole process: Feature extraction and dimensionality reduction, model induction, and inference. We describe each part in the next three subsections.

3.1 Feature Extraction and Dimensionality Reduction

The input to the model building process, namely the raw data, is the measurement logs. The logs contain the event records of all URL requests entering the system and the different operating system load metrics sampled periodically. The measurements are processed and transformed onto a time line, where for each minute, using a sliding window of one second, the state of the system is described in terms of a vector containing the following entries: statistics about each URL request (e.g, total number of instances, mean number of active instances during the minute interval, etc.), the number of active sessions, the number of arriving and departing sessions, and the mean value of the operating system metrics. The application performance in the system is recorded by noting the distribution of the response time in each window of time. Each set of measurements in the 1 minute window is the feature vector and it consists of the *workload*, the *load* and the *system response*.

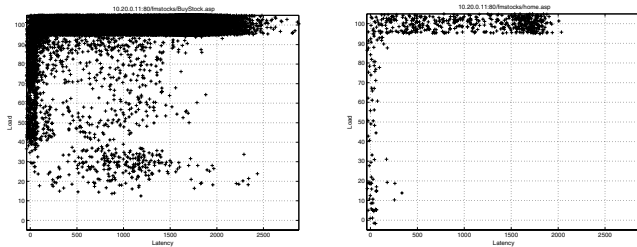


Figure 2: The histograms of two of the clusters.

Note that this set of measurements is a long vector of values. We apply dimensionality reduction techniques to address the computational complexity of such high dimensional feature spaces. To reduce the dimensionality of the work load component of this vector, we cluster the URL requests into groups according to their behavior in terms of response time required to process the URL and the av-

erage load vector of the system during its processing. To simplify the computations even further, we approximate the load with only the average CPU load of the web server during the request processing. Model selection procedures can be applied to select the most appropriate load metric.

This clustering process is not trivial and a complete and accurate description of the process is beyond the scope of this paper. We will limit ourselves to a high-level description of the algorithm and some of its main properties. Figure 2 depicts two of the resulting clusters. Note the variation along the *y*-axis which represents the CPU load. Had we not taken into account this dimension, both sets of URLs would seem to behave similarly.

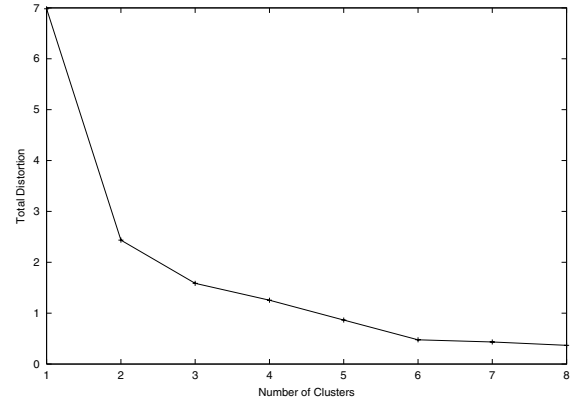


Figure 3: The total distortion as a function of the number of clusters. The elbow of the curve is between 2 and 3 clusters.

As we receive several samples for each URL, we are really clustering probability distributions, which we represent using a two dimensional histogram. The algorithm we use is based on the well known *KMeans* (see for example [3]). To completely specify Kmeans, we need to provide three things: 1) The number of clusters k ; 2) the distortion score that we intend to minimize; and 3) how we initialize the clusters. Since we are clustering over probability distributions (the two dimensional histograms), we use Kullback-Liber as a score for proximity between two probability distributions. The properties and characteristics of this score have been extensively explored in the literature of statistics, probability, and pattern recognition and the reader is referred to any of the many books on the subject [3]. Pereira et. al. [13] explored its use in clustering probability distributions in the context of text retrieval, and offers some justifications. To initialize the clusters we use the *hierarchical agglomerative clustering (HAC)* [3] algorithm. Kmeans is run several times with perturbations on the initial clusters in order to avoid local minima. To select the final number of clusters, we used the procedure described in [8]. The assumption behind this procedure is that the gain in distortion reduction grows rapidly and then asymptotically reduces its gradient to zero. The optimal number of clusters is at the elbow. To find this elbow, we fit two lines to a graph of the final distortion of Kmeans with 1, 2, 3, ... clusters and find their intersection. Figure 3 depicts this graph. Note that three is the optimal number of clusters according to this procedure. This is the one we selected for the experiments in this paper. To avoid the local minima traps, we restart the algorithm several times by perturbing the initial conditions.

3.2 Model Induction

Having described the form of the input and the feature extraction process, we are ready to describe in further detail the set of models induced. The first model we induced is comprised by a set of sessions v/s consistency (**SvC**) functions, indexed by the response time objective. Each one of this SvC functions maps a given consistency value to the number of active sessions that can be supported at that quality of service. By supported we mean, given a consistency value c , the number of active sessions such that the percentage of requests that are serviced in the server meet the response time objective is greater than or equal to c in every minute interval. Thus, the SvC function which we will denote by $C_i(c)$ directly correlates quality of service to number of sessions. An example of how the data may look for different response time objectives is depicted in Figure 4. To fit this data, we use a piecewise linear model with a set of fixed knots [5]. Figure 4 displays an example of a fitted function.

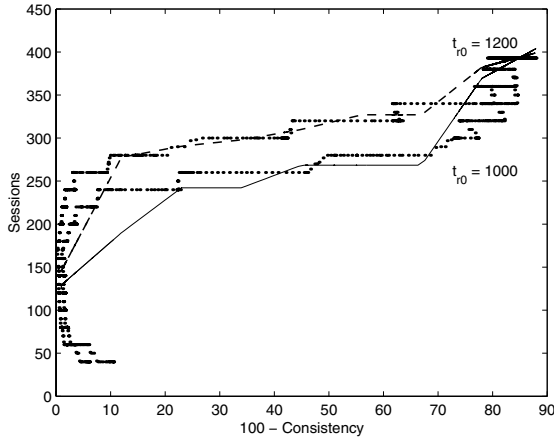


Figure 4: Sessions as a function of Consistency for different values of response time

Another approach to quantifying the notion of e-Business capacity is to implement a classifier that distinguishes the state of the system when it is in capacity to when it is not in capacity. To answer the query of if there is capacity or not, we need to have an estimate of the state of the system. However, the state of the system includes the workload and the load metrics of the different resources. We need functions that characterize the workload and the load on the resources as a function of the sessions.

To characterize the workload, we induce a second class of functions that relates the number of sessions to the requests it generates. We will use $\mathcal{S}_i(s)$ to denote each one of these functions. In this paper, we chose to have one of these functions per class of URL request (determined by the clustering routine), and also a different function for each statistical property of the request class we are trying to characterize. We use a piecewise linear model with fixed knots to fit this data.

To characterize the load metrics at the different resources, a third class of functions relates the workload to the resource load and utilization. There are many choices for selecting the model family to represent this relationship. However, in

this paper, we will focus on a multivariate regression from the different request statistics to the CPU utilization metric. We use $\vec{w}$ to denote the argument to this function and $\mathcal{L}(\vec{w})$ to denote the function itself.

Finally, we use binary classifiers to implement a version of the definition of capacity (Definition 1) above. A binary classifier is a function that, given a number of features, returns one of two values. For the purposes of this paper, these classifiers will take as input the total number of requests (in each cluster) in the one minute interval time window, the average active requests, and the average CPU load, and return a yes/no value depending whether the system is in capacity for a specific pair of values of consistency and response time objective. In fact, we induce a grid of these classifiers: one for each value of a pre-specified value of consistency and response time objective. We use $\langle \vec{w}, cpu \rangle$ to denote the input and $\mathcal{BC}_{i,c}(\vec{w}, cpu)$ for the classifier. Note that for this paper we are using only the *CPU utilization* measurements for the load vector. The methodology is equally valid for any arbitrary load vector.

There is vast literature in the statistical pattern classification on classifier induction algorithms and methods (see for example [3]). For the purposes of this paper, we explored the use of classifiers based on Bayesian networks [12, 6], and logistic regression [9]. For the Bayesian network classifier, we assume a node y that takes states from the set $\{capacity, nocapacity\}$. We call this node the *class* variable. In addition we have one node for each component of the vector $\vec{x}$. To implement the binary function $\mathcal{BC}_{i,c}(\vec{w}, cpu)$, we first induce a probability distribution $P(y, \vec{x})$ and then impose the following decision rule $\mathcal{BC}_{i,c}^{BN}(\vec{w}, cpu) = yes$ iff $P(y = capacity | \vec{w}, cpu) > P(y = nocapacity | \vec{w}, cpu)$.² In the experiments below, we assumed a fixed structure for the Bayesian network in the form of a naive Bayes model [3]. This structure and the assumption of independence it encodes enables us to compute $P(y | \vec{w}, cpu)$ as

$$P(y | \vec{w}, cpu) = \frac{\prod_i P(w_i | y) \times P(cpu | y) \times P(y)}{\sum_y (\prod_i P(w_i | y) \times P(cpu | y) \times P(y))}, \quad (1)$$

where i ranges over the length n of the vector $\vec{w}$. Note that as a consequence, for inducing the desired distribution $P(y, \vec{w}, cpu)$, we need to fit $O(n)$ independent distributions corresponding to $P(w_i | y)$, $P(cpu | y)$ and then $P(y)$. In this work, we treated all variables associated with $\vec{w}$ as continuous and we used Gaussian distributions to represent their distribution. For the class variable, we used a binomial distribution. The parameters of the distributions were fitted using a maximum likelihood approach [3]. The main reason for selecting a naive Bayes as one of the classifiers is its simplicity in terms of inference, implementation, and update. We note that for selecting the actual features in $\vec{w}$, we performed feature selection computation, with both a wrapper-based approach and a fisher discriminant-based approach [3]. We also experimented with other distribution families including multinomials based on a discretization of the features in $\vec{w}$.

For the logistic regression, we use the outcome variable as the random variable y that takes the values from the set $\{capacity, nocapacity\}$ and the input feature vector $\langle \vec{w}, cpu \rangle$

²Note that we are selecting a threshold of 0.5 for this decision rule. This threshold does influence the accuracy of the classifier and it should be set according to considerations of misclassification costs.

as in the Bayesian network classifier. The logistic regression classifier [9] fits a linear regression function to the *logit transformation*

$$\text{logit} = \log \frac{P(y = \text{capacity} | \vec{w}, \text{cpu})}{P(y = \text{nocapacity} | \vec{w}, \text{cpu})} = \beta_0 + \vec{\beta} \cdot \langle \vec{w}, \text{cpu} \rangle.$$

From the data, we first fit the regression model and apply a decision rule to implement the function $\mathcal{BC}_{l,c}^{LR}(\vec{w}, \text{cpu}) = \text{yes}$. The rule we select is $\mathcal{BC}_{l,c}^{LR}(\vec{w}, \text{cpu}) = \text{yes}$ iff $\text{logit} > 0$.¹ To model the second order interactions between the component features of $\vec{w}$, we introduce the extended feature set $\vec{w}^{\text{ext}}$ that includes the square (w_i^2) and the cross product terms ($w_i \cdot w_j$).

3.3 Inference

We are now in a position to describe in greater detail each one of the two algorithms introduced above to compute EEC. We also address the issue of how to combine the results of these algorithms (in a principled way).

The **CR** algorithm requires a simple application of the appropriate capacity function. Given a response time objective l and a consistency specification c , we simply use $\mathcal{C}_l(c)$ to compute the maximum number of sessions that can be supported in the system. Note that this is an expected value for the number of sessions. The computation of variances and errors as well as confidence intervals can be obtained in closed form. As we will explain below, these will be used in order to fuse the inferences from the second algorithm which we introduce next.

The **CBS** algorithm works as follows. Given a response time objective and consistency specification we select the appropriate classifier $\mathcal{BC}_{l,c}$. Let s denote the number of sessions. Then, using the set of functions $\mathcal{S}_i(s)$, we can reconstruct the workload vector $\vec{w}$ and, using this and $\mathcal{L}$, we compute the expected CPU utilization. The steps of the algorithm are:

1. let $s = -1$
2. repeat
 - (a) let $s = s + 1$
 - (b) Using the set of functions $\mathcal{S}_i(s)$ compute the workload vector $\vec{w}$, and then using $\mathcal{L}(\vec{w})$ compute the expected load cpu
- item until $\mathcal{BC}_{l,c}(\vec{w}, \text{cpu}) = \text{no}$
3. return $(s - 1)$

When $\mathcal{BC}_{l,c}(\vec{w}, \text{cpu}) = \text{no}$, then the boundary of capacity was crossed and the number of sessions that can be supported by the system while maintaining the quality of service is determined. In essence, the algorithm simulates a “walk” in feature space searching for the boundary of the system’s capacity.

The final question we address in this section is that of how to combine the results of the two algorithms. Before, that we briefly comment on the actual need for different algorithms for computing the effective capacity. The approach we are proposing is based on inducing models directly from observing the behavior of the system. This approach has many advantages from the point of view of adaptability and flexibility. Yet, it has to contend with uncertainty coming

from “noisy” measurements, and assumptions necessary to effectively induce models from the data.

As can be seen from Figure 4, the data used to induce the capacity function $\mathcal{C}_l$ is very nonlinear and “noisy.” Even though we are using a “flexible” approach such as a piecewise linear fitting, there will be an error in the predictions from these models, which will be reflected in the results from **CR**. The **CBS** algorithm tries to correct the sources of error by fitting a set of specialized binary functions (high bias but low variance) one for each value of response time and consistency. These functions are induced on data that is closer to the actual workload in the system. Still, there are inherent assumptions in each type of classifier and that is the main reason that we explored two types of classifiers. Bayesian networks belong to the class of “generative models” while logistic regression belongs to the class of “discriminative models.” A discussion of the trade-offs goes beyond the scope of this paper, but the differences will be made apparent in the next section. We also remark that, in addition, the error in the estimates of this algorithm is introduced by the functions $\mathcal{S}_i$ and $\mathcal{L}$.

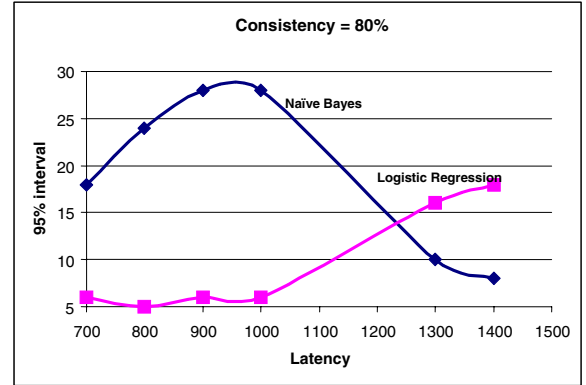


Figure 5: The width of the 95% confidence interval as a function of response time.

The general approach we take to combine the results of the different algorithms (and the different instantiation of the classifiers within the algorithms) is to compute a maximum likelihood estimate from the respective inferences. Briefly, the approach is summarized as follows: Consider a set of functions $\{f_i\}$, $1 \leq i \leq n$, such that given an input vector $\vec{x}$, each $f_i(\vec{x}) = v_i + \epsilon_i$, where v_i is a real value and ϵ_i represents noise. For this work, we assume that ϵ_i obeys a normal distribution with zero mean and sigma σ_i . Given an input $\vec{x}$ and the set $\{f_i\}$ we are interesting in computing a v^* which is the fusion of the set $\{f_i(\vec{x})\}$, and is optimal in some sense, and some characterization of the error in v^* . To this end, we assume a set of random variables r_i , each one associated with its respective v_i . These variables will obey a normal distribution with mean v_i and sigma σ_i . Each r_i provides evidence for v^* . We further assume that each variable r_i is conditionally independent from each other given the value of v^* . Under this condition, our objective is to find the value of v^* and the corresponding σ^* that maximize likelihood.

After writing the likelihood equations and performing the

maximization, the expression for v^* is:

$$v^* = \sum_i \frac{\bar{\sigma}^2}{\sigma_i^2} \times v_i, \text{ where } \frac{1}{\bar{\sigma}^2} = \sum_i \frac{1}{\sigma_i^2}, \text{ and } \sigma^{*2} = n \cdot \bar{\sigma}^2 \quad (2)$$

By the results of the Gauss-Markov theorem [2], these equations represent the unbiased estimates of minimal variance.

Note that to use these equations, we need to compute the σ of the respective inferences. This computation is possible in closed form for the case of the **CR** algorithm, but not for the case of the **CBS** algorithm. In order to compute the σ value of an application of **CBS** with both a Naive Bayes and a logistic regressor, we rely on the bootstrap method. This method relies on re-sampling the initial data a number of times and repeating the desired computation with each new data set. A complete description of this method is well beyond the scope of this paper and the interested reader is referred to [4, 7].

3.4 Results

We present some results from the application of the approach outlined above. We divide these results in three groups. The first set of results pertain to the quality of the implementation of the definition of capacity. The second illustrates the need and benefits of the fusion of the different inferences. Finally, the third group assesses the quality of the whole approach by first computing the EEC of the system and then verifying the accuracy of that prediction. For all these experiments, the setting was the following: Using a set of load generators, we simulated a web-based financial application that consists of stock buying and selling using Microsoft FMStocks application. The web system simulating the site of operation is composed of a web server, a database server, and an application server. We then increased the number of sessions steadily from 10 to 300. This was equivalent to having 4000 to 20000 requests. Part of the input data is depicted in Figure 4. Models were induced for latencies that range from 700 milliseconds to 1500 milliseconds (in increments of 100 milliseconds), and for consistencies in the set $\{2, 5, 8, 10, 12, 15, 18, 20, 22, 25, 28, 30\}$.

Accuracy of Naive Bayes. In this section, we want to provide the reader with an idea of the quality of the classifier functions as they implement the definition of capacity. The accuracy of the classifier was determined in the usual way [6]. We trained/induced each classifier with a portion of the data, called the train-set, and tested its accuracy on unseen (by the classifier) data call test-set. The accuracy is determined by the fraction of time (in terms of probability) that the classifier prediction coincides with the data. The results are depicted in Table 2. Note that for consistencies of 98% and 90%, the accuracy, averaged over all latencies, is in the low 90's with low variance. The same is true for the accuracy at the response time of 0.1sec averaged over all consistencies. In order to investigate where the classifier fails in its prediction, we also computed the probability of the classifier missing a prediction of capacity, which is in the high 70's. This indicates that the Naive Bayes classifier is conservative in the sense that it will rarely predict capacity when there is none (the results here were in the high 90's).

Inference fusion. To illustrate the effects of the fusion on inference and the need for different types of classifiers, we display in Figure 5 the difference in the standard error, with 95% confidence, between the high and low bound in the EEC computation by **CBS** for a consistency of 80%.

	Accuracy	$P(pred = Y Y)$
At consistency = 98%	92% $\pm$ 0.03	79.7% $\pm$ 0.4
At consistency = 90%	91% $\pm$ 0.2	80% $\pm$ 0.6
Response Time = 1s	91% $\pm$ 0.01	77% $\pm$ 0.1

Table 2: Accuracy results for the Naive Bayes Classifier.

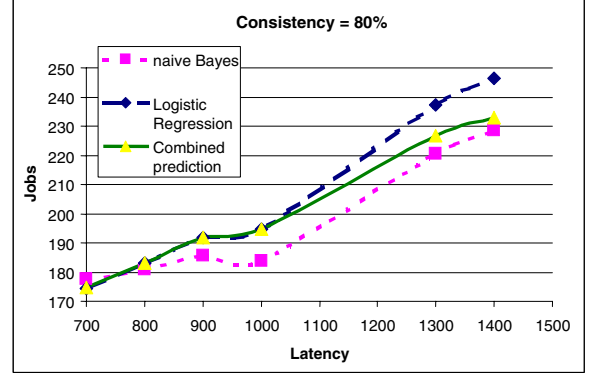


Figure 6: Fusion of the estimate of each individual classifier.

Figure 6 depicts the values of EEC returned by **CBS** as well as the final value that results from the fusion of these inferences. Note that the logistic regression is more robust to variations in the data for lower latencies and less robust at higher latencies. This justifies our belief that these classifiers introduce different assumptions and complement each other. Also note how the final result of the fusion adapts and approximates the inference with less variability.

Accuracy of EEC. In this final set of experiments, we evaluate the overall accuracy of the approach to compute EEC. The steps were as follows: First, we trained the system. Namely, we observed the system behavior when a workload such as the one described in the beginning of this subsection was applied to a three-tier infrastructure (consisting of a web server, an application server, and a database). This step took 45 minutes (of real time). Then, we induced the set of models described in Section 3.2, perform the inferences described in Section 3.3 and computed the *predicted* number of sessions or *pns* for each value of consistency and response time (from the set in the introduction of this subsection). For the third step, we subjected the system to a load, similar to the one used in the training process. Namely, one increasing from 10 sessions to 300 sessions, in increments of 10 sessions, except that we applied each load group for 30 minutes to the system. This allows the system to stabilize before we make any changes. At each stable operating state we took measurements regarding the quality of service for each minute. Finally, we computed the *actual* number of sessions or *ans* as follows: Given a response time tr the *ans* for a given consistency c , is the minimum number of sessions at a consistency c' , such that $c' \leq c$. Note that this is a worst-case scenario for defining *ans*. To compute the error, we use the formula $\frac{pns - ans}{pns} * 100$. In Table 3 we show the results for a response time of one second. Note that the error does not exceed 15%. This was true for all response times ranging from 800 milliseconds to 1500 milliseconds. At the lower response times the *pns* was more optimistic (namely it predicted more sessions than the actual number)

and at the higher response times it was more pessimistic. In comparison, other planning methodologies commonly have errors in the 30% range [10].

Consistency	Predicted	Actual	Error
98%	239	216	9.6%
96%	246	238	3.2%
95%	250	238	4.8%
94%	253	238	5.9%
92%	261	247	5.4%
90%	268	248	7.5%
82%	275	248	9.8%
86%	282	248	12.0%
85%	286	258	9.8%
84%	290	258	11.0%
82%	297	258	13.1%
80%	304	258	15.1%

Table 3: Accuracy results for the Naive Bayes Classifier.

4. APPLYING EEC

The first obvious application of EEC is in providing a global picture of the capabilities of a web site in terms of the trade-off between consistency, response times, and the number of sessions, or e-Business jobs, that the site can support. Using the algorithms described in Section 3 and smoothing the results using monotonicity constraints,³ we generate graphs such as the ones depicted in Figure 7. The user can then perform many different types of analysis for capacity planning and provisioning. For example, if-then questions about the number of e-Business jobs that a site can support at different quality of service levels.

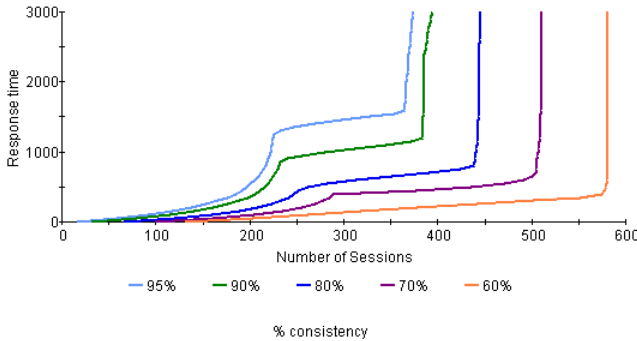


Figure 7: Fundamental behavior of a typical web site. Observe the trade-offs between response time, consistency and the number of sessions (e-Business transactions) the site can support.

Note that we can easily extend the algorithms and computations to handle different classes of services (generating different workload mixtures). We only need to fit/induce different sets of $\mathcal{S}_i$ functions relating the different services to the characteristics of the workload. As in any other business, e-Businesses must trade off one service for another. If

³Essentially we expect that as we relax the constraints on capacity and response time, the systems ability to service sessions should not decrease.

service A becomes overloaded, it not only impacts the performance of A , but also impacts service B (which shares resources with A). The EEC provides a quantitative method to evaluate this trade-off. Figure 8 demonstrates the trade-off. As we can see in the figure, we can define a *capacity boundary* which is the combination of the e-Business transactions for the different services at which the capacity reaches 100%. This boundary is represented implicitly by the functions described in Section 3.2. The non-linear nature of this boundary makes the trade-off question more interesting. If we increase the number of transactions of each service, while keeping the ratio between the number of transactions for the services constant, we march along one of the radial lines in the figure. As we change the ratio of the mixture of the two services, the point at which the boundary is crossed is different. This means that the effective capacity of the system is different as we cross the capacity boundary at the different parts of the space. Using the relative business value per transaction of the different services, we can find the optimal mixture of e-Business transactions for the different services that can be supported in the system. Note that this is a small variation on the **CBS** algorithm.

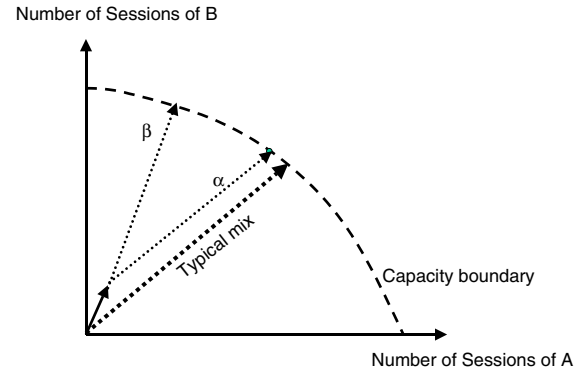


Figure 8: Trade-Off between the two services for using the effective e-Business capacity of the site.

Another variation on the **CBS** algorithm can be used to report on the available headroom or unused capacity at runtime. Given the actual number of sessions in the system and using the $\mathcal{S}_i$ and the $\mathcal{L}_i$ functions we can determine the current *state* of the system, and then find the “distance” to the capacity boundary by moving in this space in a manner similar to **CBS**. This utilization can be transformed into a percentage and information can be collected overtime as depicted in Figure 9.

Finally, we can also use EEC to correlate the utilization levels of the different components at the different tiers with the capacity utilization percentage. In this manner, we can discover bottlenecks or set up alarms in terms of metrics provided by the operating systems and databases.

5. DISCUSSION

We have described in this paper a unique and innovative approach to quantify the effective capacity of a web system. This approach builds statistical models from observed workload and performance data. This contrasts with more traditional approaches based on analytical performance models and/or simulation methods with strong roots in queuing

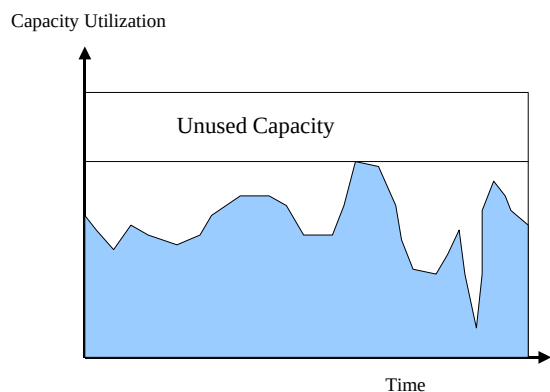


Figure 9: The effective e-Business capacity utilization of the site over time.

theory. There are advantages and disadvantages to either of these approaches. We list and discuss the most important ones. In our proposal, models are automatically and dynamically built from measurement data. No expertise is needed from the user. In our experiments, for example, the data collection lasted 45 minutes, and the programs that analyze the data and generate all the models ran for about an hour. At the end, we had induced on the order of hundreds of models characterizing the entire range of operation of the infrastructure. Note that, of course, the application of these techniques is also independent of the specific configuration of the infrastructure. This should be contrasted with the requirements of more traditional approaches involving knowledge elicitation, model construction, parameter specification, validation, and various simulations. Also, we are able to handle guarantees that contain different degrees of consistency. While analytical models based on queuing networks are good and efficient at obtaining average values, they cannot deal with percentiles. Simulation-based approaches can be used for that purpose but building and validating these models is not a trivial task.⁴ On the other hand, if an analytical model is available and validated for the infrastructure, its power is undeniable for answering questions such as the impact on performance if an individual component characteristic (disk or CPU) is changed. We could associate the performance of the actual component to the EEC, and then extrapolate based on the models induced, but more research and experimentation needs to be performed in order to assess bounds and errors. In addition, as for any statistical based approach, there are issues with being able to collect data that reflects the whole range of operation of the system (we remark that our approach is implemented so that the data can be collected in batches and all models can be updated dynamically). This should be contrasted with the requirements of an analytical model, where the service demands of each type of request for each device must be characterized. A very interesting area of research is one that pursues ways in which these two approaches can be combined so that the analytical models can bias and inform the statistical induction and pattern recognition processes.

The application of statistical machine learning algorithms to system characterization and performance opened a set of

questions for machine learning and statistical pattern recognition. Issues regarding the dynamic update of these models, characterization of errors and bounds, smoothing of results, and characterization of the regions of (feature) space where the models are accurate. These issues are beyond the scope of this paper and will be reported elsewhere.

Finally, the other contribution of this paper is the notion of effective e-Business capacity (EEC). This notion maps the business service quality objectives, at the application level, to the number of sessions that a web site can support. As illustrated in this paper, EEC can be used to formalize processes for e-Business practices, including making decisions about infrastructure (purchasing and resources allocation), perform capacity planning, pricing, and for establishing quality of service objectives.

Acknowledgments

D. Banard implemented the logistic regression algorithm. V. Volchegurski implemented the piecewise linear fitting procedure, and helped with the experiments. S. Pryce-Jones also helped with the experiments and data collection. Finally, we also wish to thank two anonymous referees for helpful comments that greatly improved the presentation of this material.

6. REFERENCES

- [1] C. Bishop. *Neural Networks for Pattern Recognition*. Morgan Kaufmann, San Mateo, 1995.
- [2] M. DeGroot. *Probability and Statistics*. Addison-Wesley, 1989.
- [3] R. O. Duda and P. E. Hart. *Pattern Classification and Scene Analysis*. Wiley, New York, 1973.
- [4] B. Efron and R. Tibshirani. *An Introduction to the Bootstrap*. Chapman & Hall, 1993.
- [5] J. Ertel and E. Fowlkes. Some algorithms for linear spline and piecewise multiple linear regression. *Journal of the American Statistical Association*, 71(355):640–648, 1976.
- [6] N. Friedman, D. Geiger, and M. Goldszmidt. Bayesian networks classifiers. *Machine Learning*, 29:131–163, 1997.
- [7] N. Friedman, M. Goldszmidt, and A. Wyner. Data analysis with bayesian networks: A bootstrap approach. In *Proceedings of Uncertainty in AI*, 1999.
- [8] M. Goldszmidt and B. Sabata. Clustering probability distributions for characterizing urls. Technical Report TR-03, Peakstone Corporation, 2001.
- [9] D. Hosmer and S. Lemeshow. *Applied Logistic Regression*. Wiley Interscience, 1989.
- [10] D. Menasce and V. Almeida. *Capacity Planning for Web Performance*. Prentice Hall, 1998.
- [11] L. Oliver. *The Cost Management Toolbox*. AMACOM, American Management Association, NY, 1990.
- [12] J. Pearl. *Probabilistic Reasoning in Intelligent Systems: Networks of Plausible Inference*. Morgan Kaufmann, San Mateo, 1988.
- [13] F. Pereira, N. Tishby, and L. Lee. Distributional clustering of English words. In *Proceedings of ACL-93*, June 1993.

⁴Daniel Menasce [10], personal communication.